SPURVANCE LABS | CYBERSECURITY

Session 1 Assignment Report

OS & Networking Fundamentals (Part 3: Practical)

Field	Details
Name	Muhammad Saad Shakeel
Session	Session 1: OS & Networking Fundamentals
Date	11 October 2026
Tasks	OSI model, Subnetting, 10 Linux commands, DNS/HTTP in Wireshark

Table of Contents

Table of Contents 2
Task 1: Draw the OSI Model with Examples 6
1.1 Overview 6
1.2 OSI Diagram 6
1.3 Layer-by-Layer Detail 6
1.4 Encapsulation Example: 7
1.5 OSI vs TCP/IP Model 8
Task 2: Subnetting Practice Sheet 9
2.1 Key Concepts and Formulas 9
2.2 CIDR / Subnet Mask Reference Table 9
2.3 Private IP Ranges (RFC 1918) 9
2.4 Practice Problems with Solutions 10
Task 3: Demo of 10 Linux Commands 12
Summary Table 12
3.1 pwd (Print Working Directory) 12
3.2 ls -la (List files) 12
3.3 mkdir and touch 12
3.4 cat and echo 13
3.5 chmod (Change permissions) 13
3.6 grep (Search text) 13
3.7 ps aux (Processes) 13
3.8 ip a (Network interfaces) 13
3.9 ping (Connectivity test) 14
3.10 ss -tuln (Sockets / open ports) 14
3.11 Bonus: dig / nslookup 14
3.12 Other Useful Commands (Quick Reference) 14
Task 4: Identify DNS and HTTP Packets in Wireshark 15
4.1 Objective 15
4.2 Tools and Setup 15
4.3 Steps Performed 15
4.4 Useful Display Filters 16
4.5 DNS Packet Analysis 16
4.6 HTTP Packet Analysis 17
4.7 Layers Seen in a Single Packet (OSI Mapping) 18
4.8 Findings Summary 18
Task 5: Python Network Sniffer using Scapy 19

5.1 Objective 19
5.2 Tools and Setup 19
5.3 Steps Performed 19
5.4 How the Code Works 22
5.5 Findings Summary 22
Task 6: Home Network Design in Cisco Packet Tracer 24
6.1 Objective 24
6.2 Network Diagram 24
6.3 Devices Used 24
6.4 Cabling and Switch Ports 25
6.5 IP Addressing Plan 25
6.6 Description of the Topology 26
6.7 How Traffic Flows 26
6.8 OSI Layer Mapping 27
6.9 Security Observations 27
6.10 Findings Summary 27
Task 7: Capture and Analyze Traffic in Wireshark 28
7.1 Objective and Setup 28
7.2 TCP Three-Way Handshake (tcp.flags.syn == 1) 28
7.3 DNS Queries and Responses (dns) 29
7.4 ICMP Echo Request and Reply (icmp) 30
7.5 Comparison of the Captured Protocols 30
7.6 Findings Summary 31
Task 8: Nmap Scan of My Own Lab Machine 32
8.1 Objective 32
8.2 Tools and Setup 32
8.3 What Nmap Does and How to Read Port States 32
8.4 Steps Performed 33
8.5 Results: Port-by-Port Analysis 35
8.6 Analysis: What the Results Show 35
8.7 Security Observations and Hardening 36
8.8 Findings Summary 36
Task 9: Wazuh SIEM Deployment (Proposed Design) 38
9.1 Objective 38
9.2 What is Wazuh? 38
9.3 Architecture 38
9.4 Proposed Lab Environment 38
9.5 Planned Deployment Steps 39

9.6 Planned Use Cases and Tests....39
9.7 Evidence to Collect Once Deployed....40
9.8 Limitations....40
9.9 Findings Summary....40
Conclusion....41

Task 1: Draw the OSI Model with Examples

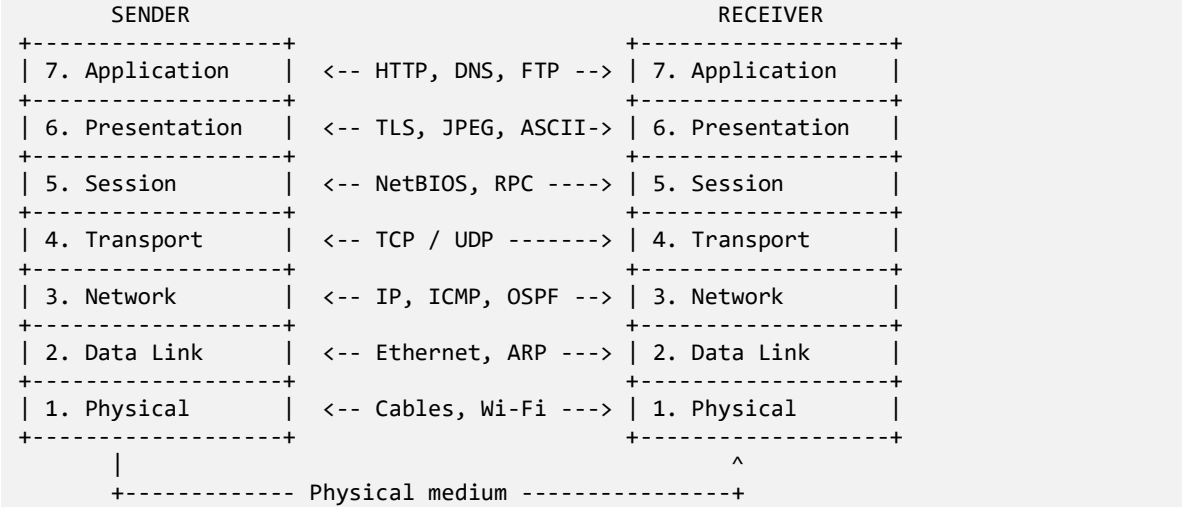
1.1 Overview

The **OSI (Open Systems Interconnection) model** is a 7-layer conceptual framework by ISO that standardises how network communication works. Each layer has a specific job, talks to the layer above and below it, and adds or removes its own header as data moves through the stack.

Memory trick (top to bottom): **A**ll **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing.

Memory trick (bottom to top): **P**lease **D**o **N**ot **T**hrow **S**ausage **P**izza **A**way.

1.2 OSI Diagram



Data flow: going down the stack is encapsulation (sender). Going up the stack is decapsulation (receiver).

1.3 Layer-by-Layer Detail

#	Layer	Function	PDU	Protocols	Devices	Example
7	Application	Interface between user apps and the network; network services	Data	HTTP, HTTPS, DNS, FTP, SMTP, POP3, IMAP, SSH, Telnet, DHCP, SNMP	Gateways, L7 firewalls, WAFs	Browser requesting a web page, email client sending mail
6	Presentation	Data translation, encryption/decryption, compression, encoding	Data	TLS/SSL, JPEG, GIF, MPEG, ASCII, Unicode, MIME	None specific	HTTPS encryption, image/video compression
5	Session	Establishes, manages and terminates sessions; checkpointing	Data	NetBIOS, RPC, PPTP, SMB sessions, SIP	None specific	Login session staying open while using a web app

#	Layer	Function	PDU	Protocols	Devices	Example
4	Transport	End-to-end delivery, segmentation, flow and error control, ports	Segment (TCP) / Datagram (UDP)	TCP, UDP, SCTP	Firewalls, load balancers	TCP for web/email (reliable), UDP for streaming/DNS/VoIP (fast)
3	Network	Logical addressing and routing between networks	Packet	IPv4, IPv6, ICMP, IPsec, OSPF, BGP, RIP	Routers, L3 switches	Router forwarding by IP; ping uses ICMP
2	Data Link	Node-to-node delivery in the same network, MAC addressing, framing, error detection	Frame	Ethernet, ARP, PPP, HDLC, 802.1Q, 802.11	Switches, bridges, NICs	Switch forwarding frames by MAC address
1	Physical	Transmission of raw bits over a medium	Bits	RJ45, Cat5e/6, fibre, 802.11 radio, Bluetooth, DSL	Hubs, repeaters, cables, modems, APs	Signals through Ethernet cable, radio waves for Wi-Fi

Sub-layers of Layer 2

- **LLC (Logical Link Control):** interfaces with the Network layer, flow control.
- **MAC (Media Access Control):** controls access to the physical medium, MAC addresses.

1.4 Encapsulation Example:

Step	Layer	What happens
1	Application	Browser creates an HTTP GET request. DNS resolves example.com to an IP.
2	Presentation	Data encoded (and encrypted if HTTPS).
3	Session	Session established/maintained with the web server.
4	Transport	TCP adds source port (random high port) and destination port 80 (443 for HTTPS). Makes a segment.
5	Network	IP adds source and destination IP. Makes a packet.
6	Data Link	Adds source/destination MAC (next hop = default gateway) plus FCS trailer. Makes a frame.
7	Physical	Frame converted to bits (electrical, light or radio signals) and sent.

The receiver reverses this (decapsulation) from Layer 1 up to Layer 7.

1.5 OSI vs TCP/IP Model

OSI Layer	TCP/IP Layer
7 Application	Application
6 Presentation	Application
5 Session	Application
4 Transport	Transport
3 Network	Internet
2 Data Link	Network Access (Link)
1 Physical	Network Access (Link)

Task 2: Subnetting Practice Sheet

2.1 Key Concepts and Formulas

- IPv4 address = 32 bits, written as 4 octets (e.g. 192.168.1.10).
- CIDR notation /n = number of network bits. Host bits = 32 - n.
- Total addresses per subnet = 2^(host bits).
- Usable hosts = 2^(host bits) - 2 (minus network and broadcast addresses).
- Number of subnets = 2^(borrowed bits).
- Block size (magic number) = 256 - (mask value in the interesting octet).
- Network address = first address in the block. Broadcast = last address in the block.
- First usable = network + 1. Last usable = broadcast - 1.

2.2 CIDR / Subnet Mask Reference Table

CIDR	Subnet Mask	Host Bits	Total Addresses	Usable Hosts
/8	255.0.0.0	24	16,777,216	16,777,214
/16	255.255.0.0	16	65,536	65,534
/19	255.255.224.0	13	8,192	8,190
/23	255.255.254.0	9	512	510
/24	255.255.255.0	8	256	254
/25	255.255.255.128	7	128	126
/26	255.255.255.192	6	64	62
/27	255.255.255.224	5	32	30
/28	255.255.255.240	4	16	14
/29	255.255.255.248	3	8	6
/30	255.255.255.252	2	4	2
/31	255.255.255.254	1	2	2 (point-to-point, RFC 3021)
/32	255.255.255.255	0	1	1 (single host)

2.3 Private IP Ranges (RFC 1918)

Class	Range	Default CIDR
A	10.0.0.0 to 10.255.255.255	/8
B	172.16.0.0 to 172.31.255.255	/12
C	192.168.0.0 to 192.168.255.255	/16

2.4 Practice Problems with Solutions

Problem 1: Split a /24 into 4 equal subnets

192.168.1.0/24. Need 4 subnets.

- Borrow 2 bits (2^2 = 4), new prefix = **/26**, mask = **255.255.255.192**.
- Block size = 256 - 192 = **64**.
- Usable hosts per subnet = 2^6 - 2 = **62**.

Subnet	Network	First Host	Last Host	Broadcast
1	192.168.1.0/26	192.168.1.1	192.168.1.62	192.168.1.63
2	192.168.1.64/26	192.168.1.65	192.168.1.126	192.168.1.127
3	192.168.1.128/26	192.168.1.129	192.168.1.190	192.168.1.191
4	192.168.1.192/26	192.168.1.193	192.168.1.254	192.168.1.255

Problem 2: Split a /16 into 8 subnets

172.16.0.0/16. Need 8 subnets.

- Borrow 3 bits (2^3 = 8), new prefix = **/19**, mask = **255.255.224.0**.
- Block size in 3rd octet = 256 - 224 = **32**.
- Usable hosts = 2^13 - 2 = **8,190**.

Subnet	Network	Usable Range	Broadcast
1	172.16.0.0/19	172.16.0.1 to 172.16.31.254	172.16.31.255
2	172.16.32.0/19	172.16.32.1 to 172.16.63.254	172.16.63.255
3	172.16.64.0/19	172.16.64.1 to 172.16.95.254	172.16.95.255
4	172.16.96.0/19	172.16.96.1 to 172.16.127.254	172.16.127.255
5	172.16.128.0/19	172.16.128.1 to 172.16.159.254	172.16.159.255
6	172.16.160.0/19	172.16.160.1 to 172.16.191.254	172.16.191.255
7	172.16.192.0/19	172.16.192.1 to 172.16.223.254	172.16.223.255
8	172.16.224.0/19	172.16.224.1 to 172.16.255.254	172.16.255.255

Problem 3: Find the network of a given host

192.168.10.77/27

- Mask = 255.255.255.224, block size = 32.
- Multiples of 32: 0, 32, **64**, 96. The value 77 falls in the 64 block.
- **Network:** 192.168.10.64
- **Usable range:** 192.168.10.65 to 192.168.10.94
- **Broadcast:** 192.168.10.95

- **Usable hosts:** 30

Problem 4: Find the network of a given host

10.1.1.130/25

- Mask = 255.255.255.128, block size = 128.
- The value 130 falls in the **128** block.
- **Network:** 10.1.1.128
- **Usable range:** 10.1.1.129 to 10.1.1.254
- **Broadcast:** 10.1.1.255
- **Usable hosts:** 126

Problem 5: Choose the right prefix for a host requirement

Hosts needed	Host bits needed	Usable hosts	Prefix	Mask
50	6 (2^6 - 2 = 62)	62	/26	255.255.255.192
100	7 (2^7 - 2 = 126)	126	/25	255.255.255.128
500	9 (2^9 - 2 = 510)	510	/23	255.255.254.0
2 (point-to-point)	2 (2^2 - 2 = 2)	2	/30	255.255.255.252

Problem 6: Are two hosts in the same subnet?

192.168.5.200/26 and 192.168.5.250/26

- Block size 64: blocks are 0, 64, 128, **192**.
- 200 is in the 192 block (192 to 255). 250 is also in the 192 block.
- **Answer: Yes.** Both are in network 192.168.5.192/26, so they can talk directly without a router.

Problem 7: VLSM (Variable Length Subnet Masking)

192.168.20.0/24. Requirements: LAN-A = 100 hosts, LAN-B = 50 hosts, LAN-C = 25 hosts, WAN link = 2 hosts.

Rule: allocate **largest first** to avoid overlap.

Segment	Hosts	Prefix	Network	Usable Range	Broadcast
LAN-A	100	/25	192.168.20.0	.1 to .126	192.168.20.127
LAN-B	50	/26	192.168.20.128	.129 to .190	192.168.20.191
LAN-C	25	/27	192.168.20.192	.193 to .222	192.168.20.223
WAN	2	/30	192.168.20.224	.225 to .226	192.168.20.227

Remaining free space: 192.168.20.228 to 192.168.20.255.

Task 3: Demo of 10 Linux Commands

Summary Table

#	Command	Purpose
1	pwd	Show current working directory
2	ls -la	List all files with details and permissions
3	mkdir - touch	Create directories and files
4	cat / echo	Display and write file content
5	chmod	Change file permissions
6	grep	Search text patterns
7	ps aux	Show running processes
8	ip a	Show network interfaces and IPs
9	ping	Test connectivity (ICMP)
10	ss -tuln	Show listening ports and sockets
Bonus	dig / nslookup	DNS lookup

3.1 pwd (Print Working Directory)

```
$ pwd
/home/saad
```

Shows the full absolute path of where you currently are.

3.2 ls -la (List files)

```
$ ls -la
total 24
drwxr-xr-x 3 saad saad 4096 Oct 11 19:40 .
drwxr-xr-x 5 root root 4096 Oct 11 19:30 ..
-rw-r--r-- 1 saad saad  220 Oct 11 19:30 .bash_logout
drwxr-xr-x 2 saad saad 4096 Oct 11 19:40 labs
```

- `-l` = long format, `-a` = show hidden files (starting with a dot).
- Columns: permissions, link count, owner, group, size, date, name.

3.3 mkdir and touch

```
$ mkdir session1
$ cd session1
$ touch notes.txt report.txt
$ ls
notes.txt  report.txt
```

- `mkdir` creates a directory (`-p` for nested directories).
- `touch` creates an empty file or updates its timestamp.

3.4 cat and echo

```
$ echo "OSI model has 7 layers" > notes.txt
$ echo "TCP is reliable" >> notes.txt
$ cat notes.txt
OSI model has 7 layers
TCP is reliable
```

- `>` overwrites a file, `>>` appends to it.
- `cat` prints file content.

3.5 chmod (Change permissions)

```
$ chmod 755 notes.txt
$ ls -l notes.txt
-rwxr-xr-x 1 saad saad 39 Oct 11 19:45 notes.txt
```

Value	Permission
4	read (r)
2	write (w)
1	execute (x)

755 = owner rwx (7), group r-x (5), others r-x (5). Also `chmod +x file` adds execute.

3.6 grep (Search text)

```
$ grep "TCP" notes.txt
TCP is reliable
$ grep -i "osi" notes.txt            # case-insensitive
$ grep -rn "password" /etc 2>/dev/null   # recursive, line numbers
```

Very useful for log analysis in security work.

3.7 ps aux (Processes)

```
$ ps aux | head -5
USER   PID %CPU %MEM  VSZ  RSS TTY STAT START TIME COMMAND
root     1  0.0  0.1 ...                        /sbin/init
```

Shows all running processes with owner, PID, CPU and memory. Combine with grep: `ps aux | grep ssh`. Use `kill <PID>` to stop a process.

3.8 ip a (Network interfaces)

```
$ ip a
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    link/ether 08:00:27:ab:cd:ef brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.15/24 brd 192.168.1.255 scope global eth0
```

Shows interface names, MAC address (link/ether), IPv4 address with CIDR (inet), and state. This ties directly to Task 2 (subnetting) and Task 1 (Layers 2 and 3).

3.9 ping (Connectivity test)

```
$ ping -c 4 8.8.8.8
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=18.2 ms
...
4 packets transmitted, 4 received, 0% packet loss
```

Uses **ICMP** (Layer 3). `-c 4` sends 4 packets. TTL and latency show distance and speed.

3.10 ss -tuln (Sockets / open ports)

```
$ ss -tuln
Netid State  Local Address:Port
udp   UNCONN 127.0.0.53%lo:53
tcp   LISTEN 0.0.0.0:22
```

- `-t` TCP, `-u` UDP, `-l` listening, `-n` numeric. Replaces older `netstat`.
- Useful to find unexpected open ports (security auditing).

3.11 Bonus: dig / nslookup

```
$ nslookup example.com
$ dig example.com A
```

Performs a DNS lookup. We use this to generate DNS traffic for Task 4.

3.12 Other Useful Commands (Quick Reference)

Command	Use
whoami, id	Current user and IDs
uname -a	Kernel and system info
cd, cp, mv, rm	Navigate, copy, move, delete
sudo	Run as root
traceroute	Show hops to a host
curl / wget	Fetch web content
man <cmd>	Manual page
nmap	Port scanning

Task 4: Identify DNS and HTTP Packets in Wireshark

4.1 Objective

Capture live network traffic with Wireshark and identify **DNS** and **HTTP** packets, analysing their layers and key fields.

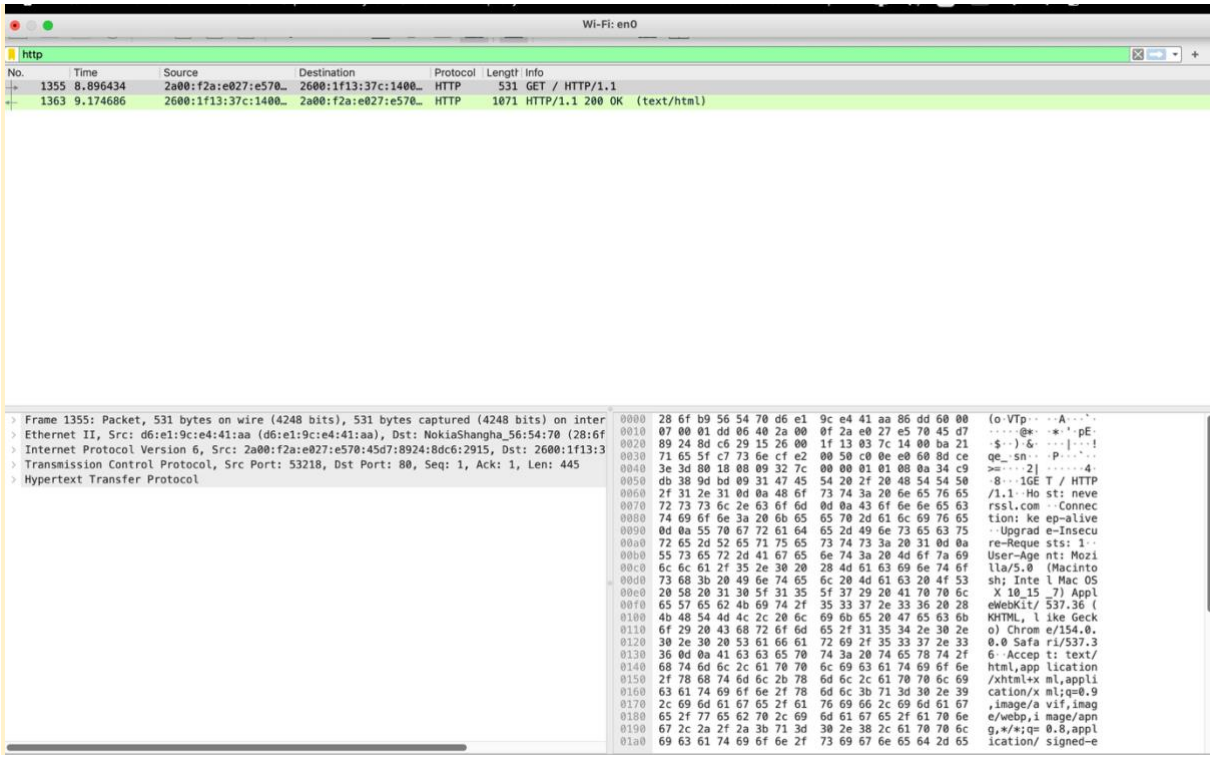
4.2 Tools and Setup

- **Wireshark** (packet analyser)
- Interface used: eth0 / Wi-Fi
- Test site: http://example.com (plain HTTP, so the content is visible. HTTPS is encrypted and would not show readable HTTP)

4.3 Steps Performed

1. Open Wireshark and select the active network interface (the one with the moving traffic graph).
2. Click **Start Capturing Packets** (blue shark fin).
3. In a terminal, generate DNS traffic: `nslookup example.com`
4. Generate HTTP traffic: `curl` http://youtube.com
5. Click **Stop** (red square).
6. Apply display filter `dns`, then analyse packets.
7. Apply display filter `http`, then analyse packets.
8. Save the capture: File > Save As (`session1_capture.pcapng`)..

Wireshark interface for HTTP



4.4 Useful Display Filters

Filter	Purpose
dns	Show all DNS packets
udp.port == 53	DNS over UDP port 53
dns.qry.name == "example.com"	DNS queries for a specific domain
dns.flags.response == 1	Only DNS responses
http	Show all HTTP packets
tcp.port == 80	Traffic on HTTP port
http.request.method == "GET"	Only GET requests
http.response.code == 200	Only 200 OK responses
ip.addr == <IP>	Traffic to or from an IP
tcp.flags.syn == 1	TCP SYN packets (handshake start)

4.5 DNS Packet Analysis

DNS translates domain names into IP addresses. It uses **UDP port 53** (TCP 53 for large responses and zone transfers).

DNS Query (client to DNS server)

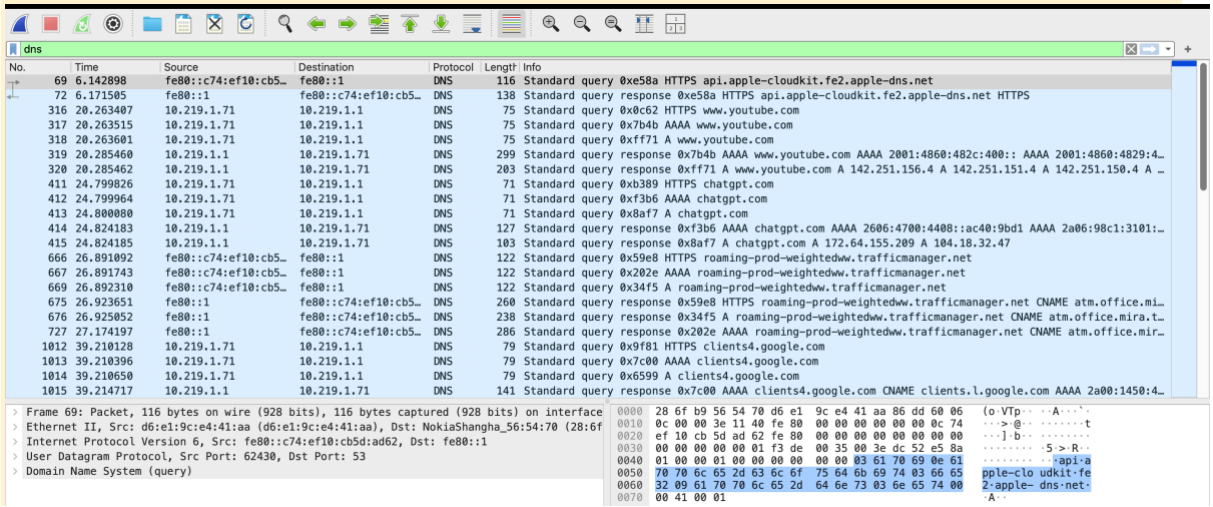
Field	Value (example, replace with yours)
Source IP	Your machine (e.g. 192.168.1.15)
Destination IP	DNS server (e.g. 8.8.8.8 or router)
Protocol	DNS over UDP
Source Port	Random high port (e.g. 54321)
Destination Port	53
Transaction ID	e.g. 0x1a2b (matches query and response)
Flags	Standard query (QR = 0)
Question	example.com, Type A (IPv4), Class IN

DNS Response (server to client)

Field	Value
Source Port	53
Flags	Standard query response, No error (QR = 1)
Answer	example.com A record, IP address of the site
TTL	Time (seconds) the answer can be cached

Common record types: **A** (IPv4), **AAAA** (IPv6), **CNAME** (alias), **MX** (mail), **NS** (name server), **PTR** (reverse), **TXT**.

Screenshot : dns filter showing query and response. And expanded DNS packet details pane.



4.6 HTTP Packet Analysis

HTTP is the web protocol at the Application layer. It runs over **TCP port 80** (HTTP) and 443 (HTTPS). Before HTTP, a **TCP 3-way handshake** happens: SYN, then SYN-ACK, then ACK.

HTTP Request (GET)

```
GET / HTTP/1.1
Host: youtube.com
User-Agent: curl/8.x
Accept: */*
```

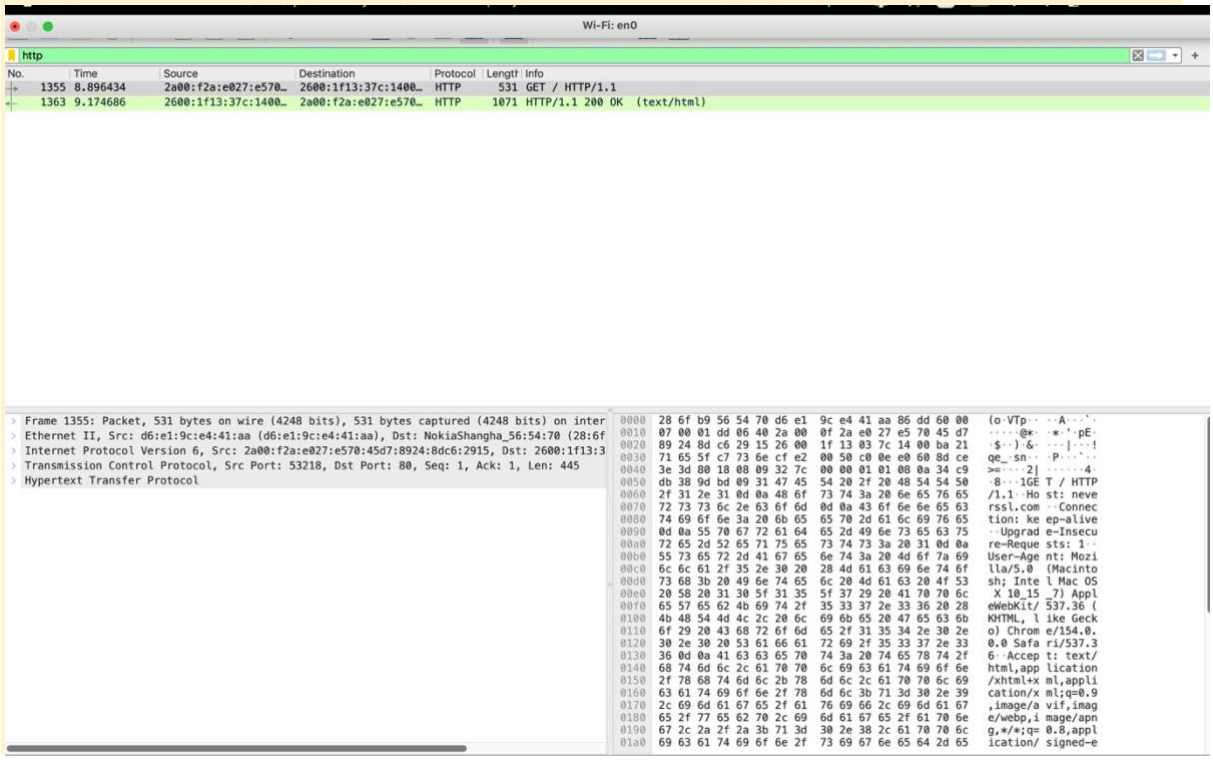
Field	Meaning
Method	GET (retrieve), POST (send data), PUT, DELETE, HEAD
URI / Path	Resource requested (/)
Version	HTTP/1.1
Host	Target domain
User-Agent	Client software/browser
Accept	Content types the client accepts

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1256
```

Field	Meaning
Status code	200 OK, 301/302 redirect, 403 forbidden, 404 not found, 500 server error
Content-Type	Type of returned data (e.g. text/html)
Content-Length	Size of body
Body	The actual HTML content

Screenshot placeholder: http filter showing GET and 200 OK.



4.7 Layers Seen in a Single Packet (OSI Mapping)

Wireshark's packet details pane maps directly to the OSI model (links back to Task 1):

Wireshark Section	OSI Layer	Key Info
Frame	Layer 1 (Physical)	Bytes on wire, capture time
Ethernet II	Layer 2 (Data Link)	Source and destination MAC
Internet Protocol v4	Layer 3 (Network)	Source and destination IP, TTL
UDP / TCP	Layer 4 (Transport)	Ports, sequence numbers, flags
DNS / HTTP	Layer 7 (Application)	Query/response, request/response data

4.8 Findings Summary

Item	DNS	HTTP
Transport protocol	UDP	TCP
Port	53	80
Purpose	Resolve name to IP	Transfer web content
Packets observed	Query + Response	TCP handshake + GET + 200 OK
Encrypted?	No (unless DoH/DoT)	No (HTTPS would be encrypted)

Task 5: Python Network Sniffer using Scapy

5.1 Objective

Build a beginner-level **network sniffer** in Python using the **Scapy** library. The tool captures live packets and displays the **source IP**, **destination IP**, **protocol**, **ports** and **packet size** for each one, like a very lightweight Wireshark.

Planned phases: **Phase 1** basic live packet display (this task), **Phase 2** protocol analysis (TCP, UDP, DNS, ICMP, HTTP), **Phase 3** security features (filters, PCAP export, suspicious traffic checks).

5.2 Tools and Setup

- **Kali Linux** running inside **UTM** on a MacBook Air M2 (8 GB RAM)
- **Python 3** and the **Scapy** library (`python3-scapy`)
- Root privileges (`sudo`) are required because packet capture needs raw socket access

Important: capture only traffic on your own device or an authorised lab network. This sniffer inspects packet metadata (addresses, protocol, ports, size) and does not collect private payloads.

5.3 Steps Performed

Step 1: Check Python and install Scapy

```
python3 --version
sudo apt update
sudo apt install -y python3-scapy
```

Scapy is a Python library that can build, capture and inspect network packets.

Step 2: Create the project

```
mkdir -p ~/python-sniffer
cd ~/python-sniffer
nano sniffer.py
```

This creates a folder named python-sniffer and opens a new file called sniffer.py in Nano.

Step 3: Write the sniffer code

Paste the full program below into Nano. Save with **Ctrl + O**, press **Enter** to confirm the file name, then exit with **Ctrl + X**.

```
from scapy.all import sniff, IP, IPv6, TCP, UDP, ICMP, ARP
from datetime import datetime

def packet_callback(packet):
    # Identify source and destination addresses
    if IP in packet:
        source = packet[IP].src
        destination = packet[IP].dst
    elif IPv6 in packet:
        source = packet[IPv6].src
        destination = packet[IPv6].dst
    elif ARP in packet:
        source = packet[ARP].psrc
        destination = packet[ARP].pdst
    else:
        return
```

```
    # Identify the protocol
    if TCP in packet:
        protocol = "TCP"
        details = (
            f"Ports: {packet[TCP].sport} -> "
            f"{packet[TCP].dport}"
        )
    elif UDP in packet:
        protocol = "UDP"
        details = (
            f"Ports: {packet[UDP].sport} -> "
            f"{packet[UDP].dport}"
        )
    elif ICMP in packet:
        protocol = "ICMP"
        details = "Ping/control message"
    elif ARP in packet:
        protocol = "ARP"
        details = "Address resolution"
    else:
        protocol = "Other"
        details = ""

    # Display packet information
    timestamp = datetime.now().strftime("%H:%M:%S")
    size = len(packet)
    print(
        f"[{timestamp}] {protocol:5} | "
        f"{source} -> {destination} | "
        f"Size: {size} bytes | {details}"
    )

print("=" * 75)
print("          PYTHON NETWORK SNIFFER")
print("          Press CTRL+C to stop")
print("=" * 75)

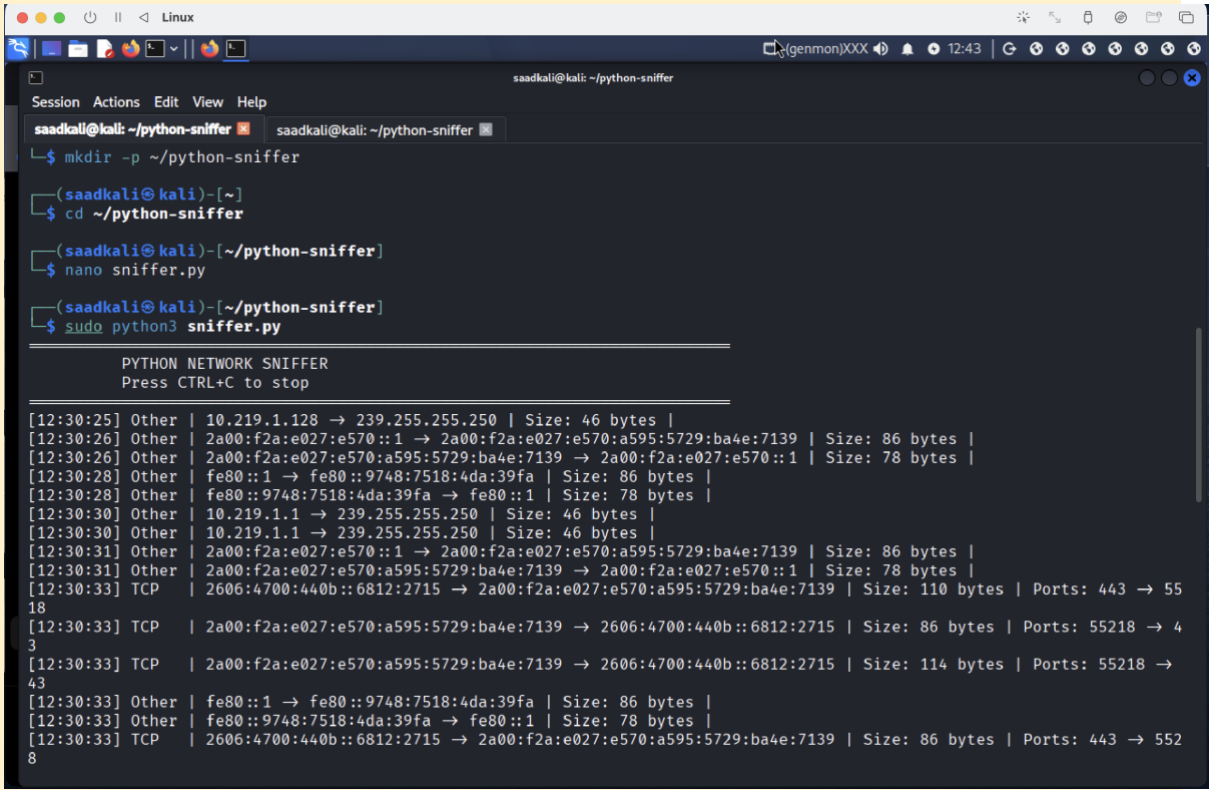
# Start capturing. store=False avoids keeping every packet in memory.
sniff(prn=packet_callback, store=False)
```

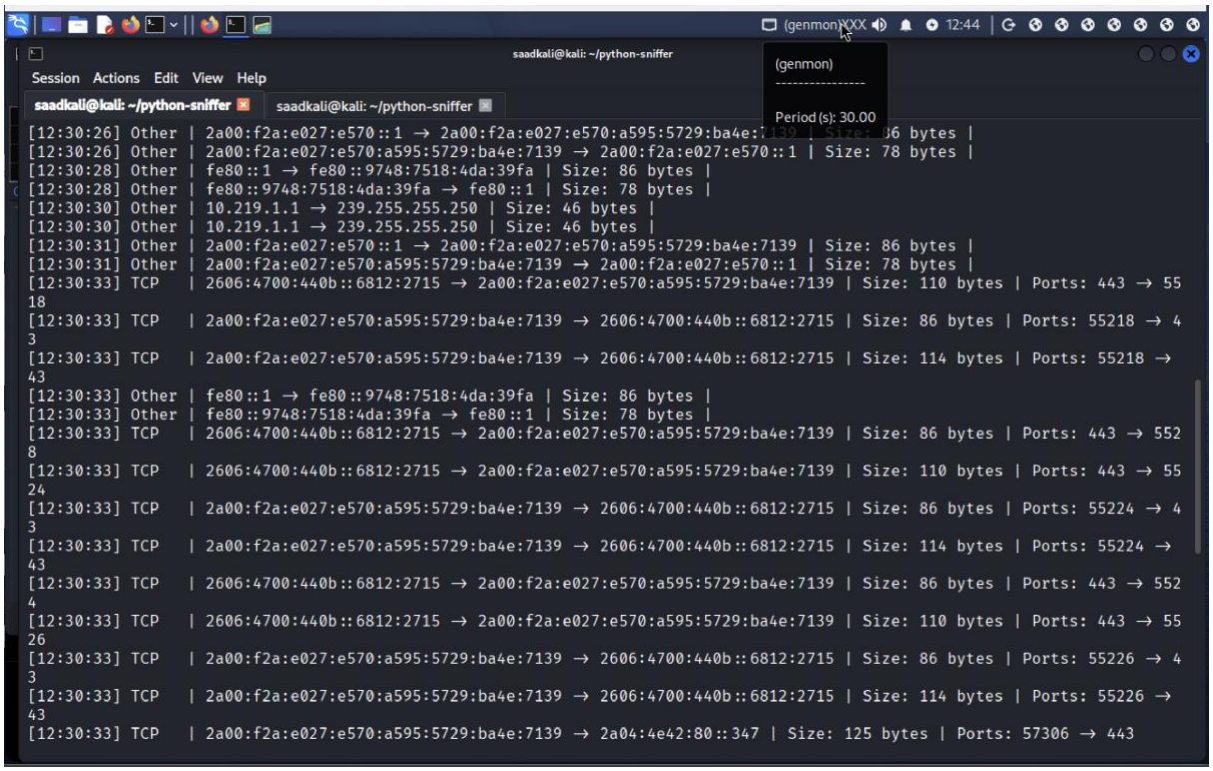
Step 4: Run the sniffer

```
sudo python3 sniffer.py
```

The program prints the banner below and then waits for packets. Waiting with no output at first is normal.

Screenshot placeholder: sniffer started in Terminal 1 showing the banner.





Step 5: Generate traffic and capture it

Open a **second terminal** in Kali and generate traffic while the sniffer is running:

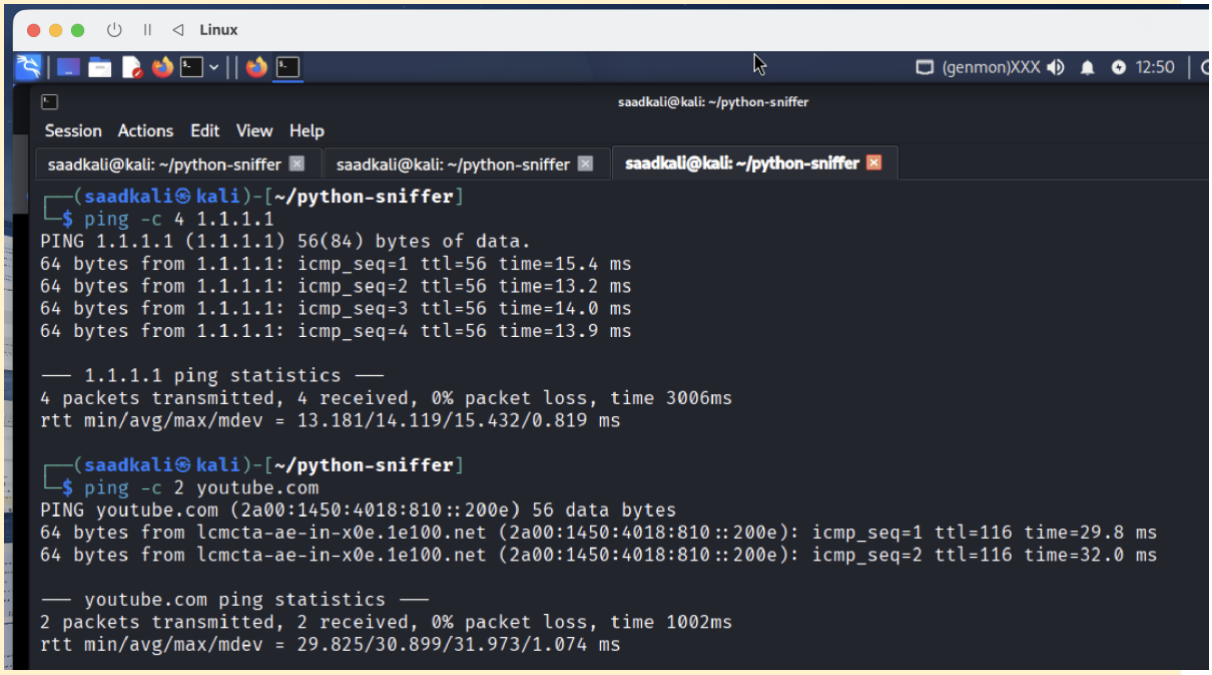
```
ping -c 4 1.1.1.1
ping -c 2 youtube.com
```

Switch back to the first terminal. ICMP packets for the ping request and reply should appear

```
[21:35:01] ICMP   | 10.0.0.2 -> 1.1.1.1 | Size: 84 bytes | Ping/control message
[21:35:01] ICMP   | 1.1.1.1 -> 10.0.0.2 | Size: 84 bytes | Ping/control message
```

IP addresses, sizes and timestamps will differ..

Screenshot placeholder: Terminal 1 (sniffer output) and Terminal 2 (ping command) side by side



.

5.4 How the Code Works

Component	Purpose
`sniff()`	Captures packets from the network and passes each one to the callback
`packet_callback()`	Processes each captured packet and prints one summary line
`IP` / `IPv6` / `ARP`	Extract the source and destination addresses
`TCP` / `UDP`	Identify the transport protocol and the source/destination ports
`ICMP`	Identifies ping-related (control) traffic
`datetime`	Adds a timestamp to every output line
`store=False`	Avoids keeping all captured packets in memory

5.5 Findings Summary

Item	Result
Milestone	Captured the packets generated by my own ping command
Protocols identified	ICMP (ping), plus TCP, UDP, IPv6 and ARP handled by the code
Information shown	Timestamp, protocol, source IP, destination IP, size, ports

Privilege needed	Root (sudo) for raw packet capture
Next upgrade	DNS filter to watch domain lookups (e.g. example.com) in real time, then PCAP export

Task 6: Home Network Design in Cisco Packet Tracer

6.1 Objective

Design and simulate a small **home network** in **Cisco Packet Tracer**, using a single switch as the central device. The aim is to show how typical home devices (computers, a server, a phone, a printer and wireless equipment) connect to one network and to the router that provides the way out of it. This links back to the **OSI model** (Task 1), **subnetting** (Task 2) and the **packet analysis** work (Tasks 4 and 5).

6.2 Network Diagram

The topology below was built in Packet Tracer. Every wired device connects directly to the central switch, so the network follows a **star topology**. The green triangles at each end of a cable are link indicators and show that the link is up.

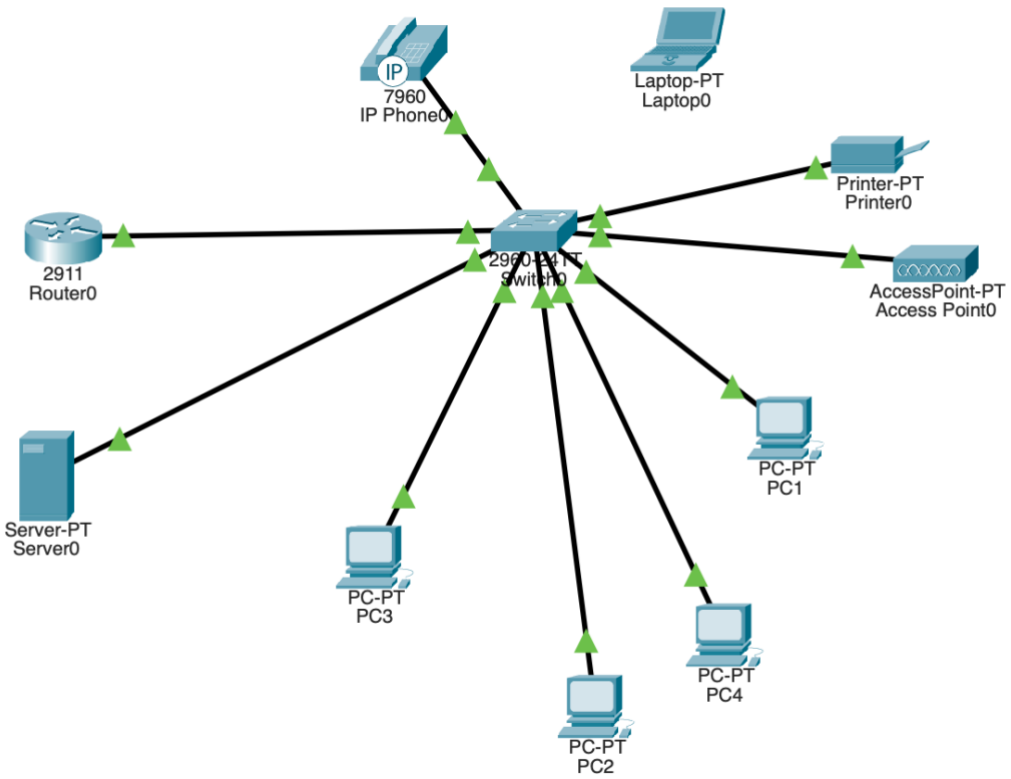


Figure 6.1: Home network topology in Cisco Packet Tracer

6.3 Devices Used

Device	Packet Tracer model	Role in the network
Switch0	Cisco 2960-24TT	Central Layer 2 switch. Forwards frames between all wired devices using MAC addresses.
Router0	Cisco 2911	Layer 3 device and default gateway. Routes traffic between the home LAN and any other network, such as the Internet.

Device	Packet Tracer model	Role in the network
Server0	Server-PT	Provides network services (for example web, DNS or file services) to the other devices.
PC1, PC2, PC3, PC4	PC-PT	End-user computers. They are the clients that request services and send traffic.
IP Phone0	Cisco 7960	VoIP phone that sends voice as IP packets over the same network.
Printer0	Printer-PT	Network printer shared by all devices on the LAN.
Access Point0	AccessPoint-PT	Provides Wi-Fi. It turns the wired network into a wireless one for mobile devices.
Laptop0	Laptop-PT	Wireless client. It is placed in the diagram without a wired cable.

6.4 Cabling and Switch Ports

Each wired device was connected to Switch0 with a **Copper Straight-Through** cable. This is the correct cable type because each link joins two different kinds of device (end device or router to switch).

Steps in Packet Tracer:

- Click **Connections** (the lightning bolt icon) in the device bar at the bottom left.
- Choose **Copper Straight-Through**.
- Click the device (for example PC1) and select its port, then click **Switch0** and select the matching switch port from the table below.
- Repeat for every wired device. Each link shows amber at first and turns green once the switch port is forwarding.

The ports used on each side of every cable are listed below:

Device	Port on device	Connected to Switch0 port
Router0	GigabitEthernet0/0	GigabitEthernet0/1
PC1	FastEthernet0	FastEthernet0/1
PC2	FastEthernet0	FastEthernet0/2
PC3	FastEthernet0	FastEthernet0/3
PC4	FastEthernet0	FastEthernet0/4
Server0	FastEthernet0	FastEthernet0/5
Printer0	FastEthernet0	FastEthernet0/6
Access Point0	Port 0 (Ethernet)	FastEthernet0/7

6.5 IP Addressing Plan

All devices share one private subnet, **192.168.1.0/24** (RFC 1918, see Task 2), with subnet mask **255.255.255.0**. Router0 holds the first address, **192.168.1.1**, and is the default gateway for every other device. Servers and network devices use fixed addresses in the higher range, while the PCs use the lowest addresses.

Device	IP address	Subnet mask	Default gateway
Router0	192.168.1.1	255.255.255.0	—
PC1	192.168.1.2	255.255.255.0	192.168.1.1
PC2	192.168.1.3	255.255.255.0	192.168.1.1
PC3	192.168.1.4	255.255.255.0	192.168.1.1
PC4	192.168.1.5	255.255.255.0	192.168.1.1
Server0	192.168.1.10	255.255.255.0	192.168.1.1
Printer0	192.168.1.20	255.255.255.0	192.168.1.1
Access Point0	192.168.1.30	255.255.255.0	192.168.1.1
IP Phone0	192.168.1.40	255.255.255.0	192.168.1.1

The IP address is entered on each device under **Desktop > IP Configuration** (or the **Config** tab on the server, printer and router). It can be checked with **ipconfig** in the PC command prompt. Laptop0 is a wireless client and is not listed because it would get its address after joining the access point.

6.6 Description of the Topology

Reading the diagram, the network is built around Switch0 in the centre:

- Router0 (192.168.1.1) connects to the switch on the left. It is the exit point of the network and acts as the default gateway for every device.
- Server0 (192.168.1.10) connects to the switch on the lower left and offers services to the rest of the LAN.
- PC1, PC2, PC3 and PC4 (192.168.1.2 to 192.168.1.5) connect to the lower half of the switch and represent the family computers.
- IP Phone0, Printer0 and Access Point0 connect to the upper half of the switch, so voice, printing and Wi-Fi all share the same wired backbone.
- Laptop0 has no cable in the diagram. As a wireless device, it would join the network by associating with Access Point0.

All the links are solid black lines, which in Packet Tracer represent copper straight-through Ethernet cables. The green link indicators show that every cable is working and that all the connected ports are up.

6.7 How Traffic Flows

Scenario	What happens
PC1 to PC3 (same LAN)	PC1 (192.168.1.2) sends an ARP request to find the MAC address of PC3 (192.168.1.4). Switch0 learns the source MAC, then forwards the frame only out of the port that leads to PC3. The router is not involved.
PC1 to Server0 (same LAN)	The same Layer 2 process applies. The switch delivers the frame directly to the server (192.168.1.10), and the reply goes back the same way.
PC1 to an outside network	The destination is not in 192.168.1.0/24, so PC1 sends the packet to its default gateway, Router0 (192.168.1.1). The router forwards it onwards and sends the replies back through the switch.
Laptop0 to the LAN	The laptop connects over Wi-Fi to Access Point0 (192.168.1.30), which passes its traffic onto Switch0 through the wired link.

6.8 OSI Layer Mapping

Device	OSI layer	Reason
Cables	Layer 1 (Physical)	Carry the raw bits between devices.
Access Point0	Layer 1 and 2	Handles radio signals and frames between Wi-Fi clients and the wired LAN.
Switch0	Layer 2 (Data Link)	Forwards frames by MAC address.
Router0	Layer 3 (Network)	Forwards packets by IP address between networks.
PCs, Server0, Phone, Printer	Layers 3 to 7	Create and consume data using IP, TCP/UDP and application protocols.

6.9 Security Observations

- **Flat network:** everything is on one switch and, by default, one VLAN and one subnet. A compromised PC or the printer could reach the server and the other devices directly. VLANs would separate users, servers and IoT or guest devices.
- **Single point of failure:** if Switch0 or Router0 fails, the whole network loses connectivity. A home network accepts this risk, while a business network would add redundant links.
- **Wireless entry point:** Access Point0 should use WPA2 or WPA3 with a strong passphrase, and guest users should be kept on a separate network.
- **Router as the perimeter:** Router0 is the only exit from the network, so it is the natural place for a firewall or ACLs and for logging traffic.
- **Monitoring:** because all traffic passes through Switch0, a SPAN (port mirroring) port could send a copy of it to a capture tool, which connects this task with the Wireshark work in Task 4.

6.10 Findings Summary

Item	Result
Topology	Star, with Switch0 (Cisco 2960-24TT) at the centre
Devices	1 switch, 1 router, 1 server, 4 PCs, 1 IP phone, 1 printer, 1 access point and 1 laptop
Cabling	Copper straight-through Ethernet, with all links up (green indicators)
Addressing	192.168.1.0/24, with devices numbered from 192.168.1.1 to 192.168.1.40
Gateway	Router0 (Cisco 2911) at 192.168.1.1
Wireless	Access Point0 for wireless clients such as Laptop0
Main risk observed	Flat network with no segmentation and single points of failure

Task 7: Capture and Analyze Traffic in Wireshark

7.1 Objective and Setup

Capture live traffic with Wireshark and analyse the main protocols seen on the network: the TCP three-way handshake, DNS and ICMP. Each capture was filtered separately and is shown below with my analysis.

- Tool: Wireshark on the Wi-Fi interface (en0)
- Traffic generated: browsing (tryhackme.com, google.com) and ping 1.1.1.1
- Display filters used: tcp.flags.syn == 1, dns, icmp

7.2 TCP Three-Way Handshake (tcp.flags.syn == 1)

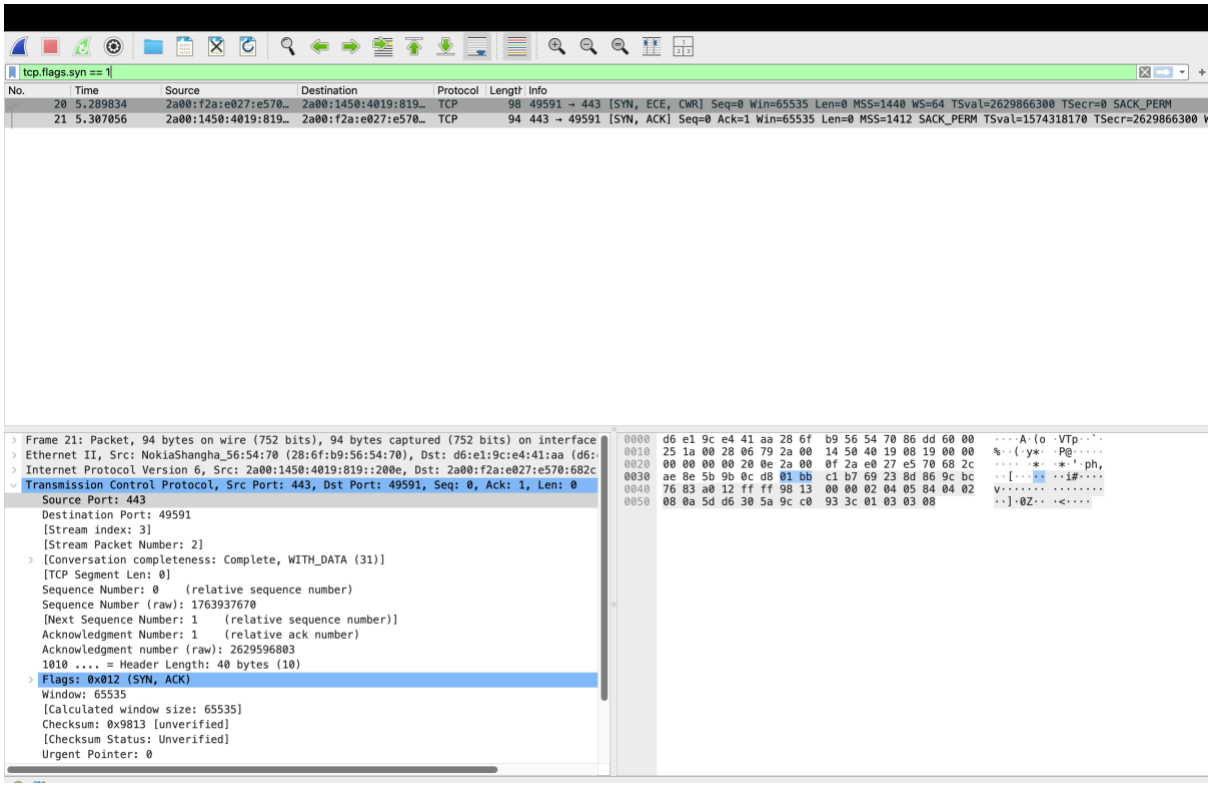


Figure 7.1: Wireshark filtered with tcp.flags.syn == 1, showing the SYN and SYN-ACK of one connection.

The filter `tcp.flags.syn == 1` shows only packets with the SYN flag set, which means the first two steps of every TCP handshake. Two packets are listed for one connection (stream index 3):

- **Packet 20 (SYN):** my machine (`2a00:f2a:e027:e570...`) sends a SYN from port 49591 to port 443 with Seq=0. It also advertises MSS=1440, window scaling (WS=64) and SACK_PERM. The ECE and CWR flags show that my machine supports congestion notification (ECN).
- **Packet 21 (SYN, ACK):** the server (`2a00:1450:4019:819::200e`) replies from port 443 with flags `0x012` (SYN + ACK), Seq=0 and Ack=1. Ack=1 means it received my SYN and expects my next byte to be number 1. It offers MSS=1412 and a window of 65535.

The SYN-ACK arrived about 17 ms after the SYN (5.289834 s to 5.307056 s), which is the round-trip time to the server. The details pane shows the TCP header length is 40 bytes because of these options, and Wireshark marks the conversation as "Complete, WITH_DATA", so the final ACK and the

data transfer also took place. Both addresses are IPv6, and port 443 means this connection was HTTPS, so the data after the handshake is encrypted.

7.3 DNS Queries and Responses (dns)

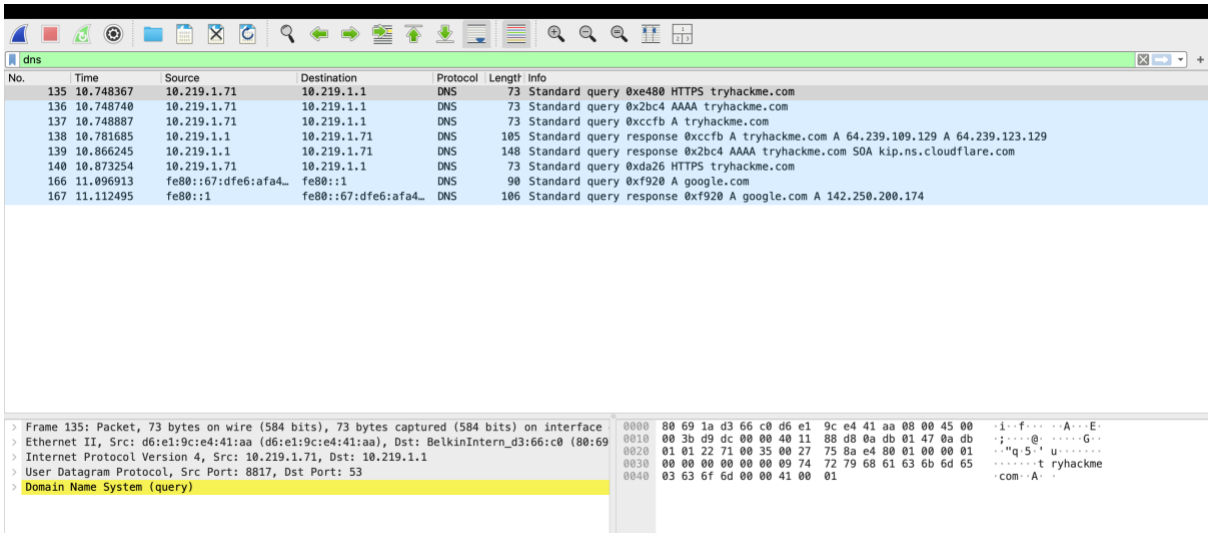


Figure 7.2: DNS queries and responses for tryhackme.com and google.com.

After opening tryhackme.com, Wireshark shows my machine (`10.219.1.71`) sending DNS queries to my router (`10.219.1.1`), which acts as the DNS resolver. In the packet details, frame 135 is 73 bytes and uses UDP with source port 8817 and destination port 53.

Packet	Type	What it shows
135, 136, 137	Queries (HTTPS, AAAA, A)	Three queries sent at almost the same moment (about 10.748 s) for tryhackme.com, with different IDs (0xe480, 0x2bc4, 0xccfb).
138	Response to A query (0xccfb)	Two IPv4 addresses returned: 64.239.109.129 and 64.239.123.129. The transaction ID matches the query.
139	Response to AAAA query (0x2bc4)	No IPv6 address in the answer. It returns an SOA record from kip.ns.cloudflare.com, so the site is behind Cloudflare and has no AAAA record.
166, 167	Query and response for google.com	Sent over IPv6 link-local addresses (fe80::). The A record answer is 142.250.200.174.

Observations: the A record is for IPv4 and AAAA is for IPv6. Modern systems send both at once. Responses are matched to queries by the transaction ID. The DNS response times were about 33 ms. All of this traffic is unencrypted UDP port 53, so anyone on the network could see which sites I visit.

7.4 ICMP Echo Request and Reply (icmp)

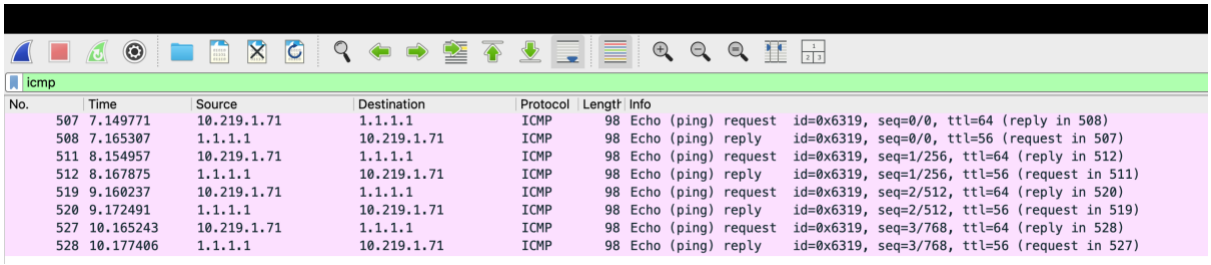


Figure 7.3: ICMP echo requests and replies between 10.219.1.71 and 1.1.1.1 (ping).

I ran `ping 1.1.1.1` (Cloudflare DNS) while capturing. The filter `icmp` shows 4 echo requests and 4 echo replies, 8 packets in total, each 98 bytes. Every request is followed immediately by its reply, and Wireshark links them ("reply in 508", "request in 507").

Field	Observation
Source / Destination	Requests go from 10.219.1.71 to 1.1.1.1 and replies come back the other way.
Identifier	0x6319 in all packets, so they belong to the same ping session.
Sequence number	0, 1, 2, 3 (shown as 0/0, 1/256, 2/512, 3/768 because Wireshark also shows the byte-swapped value).
TTL	Requests have TTL 64 (my OS default). Replies have TTL 56, so the reply passed through about 8 routers.
Round-trip time	About 12 to 16 ms per ping (for example, 7.149771 s to 7.165307 s is 15.5 ms).
Interval	Requests are sent 1 second apart, which is the normal ping default.

ICMP works at Layer 3 (Network). It has no port numbers and no TCP handshake, which is different from TCP and UDP. It is used for diagnostics such as ping and traceroute, and attackers can also abuse it for network scanning or ICMP tunnelling.

7.5 Comparison of the Captured Protocols

Protocol	OSI Layer	Transport	Key observation
DNS	7 (Application)	UDP 53	Name to IP lookup; queries and responses matched by ID; unencrypted.
HTTP	7 (Application)	TCP 80	GET request and 200 OK response in plain text.
TCP handshake	4 (Transport)	TCP 443	SYN then SYN-ACK (then ACK) before any data is sent.
ICMP	3 (Network)	None (IP)	Echo request and reply used to test reachability; TTL and RTT visible.

7.6 Findings Summary

In summary: Wireshark lets me see exactly how protocols behave at different layers. DNS and ICMP show that a lot of traffic is readable by anyone who can capture it, while the 443 connection shows that after the handshake the content is encrypted. For security work, this type of analysis helps detect suspicious DNS requests, unexpected scans and abnormal connections.

Task 8: Nmap Scan of My Own Lab Machine

8.1 Objective

Use **Nmap** (Network Mapper) to scan my **own lab machine** and find out which network services are listening on it. The aim is to see my computer the way an attacker would first see it: as a list of open ports. I then confirm what each port really belongs to using **lsof**, and check whether the service is exposed only to the machine itself or to the whole network. This links back to **ports and TCP** (Tasks 1 and 4), the **IP addressing and subnet** work (Task 2), and the **ss -tuln** command (Task 3.10), which answers the same question from inside the machine.

8.2 Tools and Setup

- **Machine scanned:** my own MacBook Air (macOS), Wi-Fi interface `en0`, IP address `10.219.1.71`
- **Nmap 7.98** run from the macOS Terminal (not the Kali VM used in earlier tasks)
- **Supporting commands:** `ifconfig`, `netstat`, `sudo lsof`
- **Target:** `127.0.0.1` (localhost, the loopback address of my own machine)

8.3 What Nmap Does and How to Read Port States

Nmap sends probes to the ports of a target and reports how each port answers. Ports work at the **Transport layer (Layer 4)** of the OSI model, so every result below is about TCP ports. Nmap reports one of three main states:

State	Meaning
open	A program is listening and accepted the connection. This is the attack surface of the machine.
closed	The port is reachable but nothing is listening. The machine actively refused the connection (conn-refused).
filtered	A firewall is dropping the probes, so Nmap cannot tell whether the port is open or closed.

Two scan modes were used in this task:

- **Default scan** (`nmap <target>`): checks the 1000 most common TCP ports and guesses each service name from a table of well-known port numbers.
- **Version detection** (`-sV`): connects to each open port, reads the reply and tries to identify the real software and version. This works at the **Application layer (Layer 7)**.

8.4 Steps Performed

Step 1: Basic port scan of localhost

```
nmap 127.0.0.1
```

This scans the 1000 most common TCP ports on my own machine. It finished in **0.10 seconds** because loopback traffic never leaves the computer (latency 0.000032 s). Nmap reported **995 closed ports** (conn-refused) and **5 open ports**: 3306, 5000, 5432, 7000 and 8021.

```
(base) saad@Muhammads-MacBook-Air ~ % nmap 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-16 16:18 +0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000032s latency).
Not shown: 995 closed tcp ports (conn-refused)
PORT     STATE SERVICE
3306/tcp open  mysql
5000/tcp open  upnp
5432/tcp open  postgresql
7000/tcp open  afs3-fileserver
8021/tcp open  ftp-proxy

Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds
```

Figure 9.1: Default Nmap scan of 127.0.0.1 showing 5 open TCP ports

Step 2: Check my own network interface

```
ifconfig en0
```

This shows the address settings of the Wi-Fi interface, so I know which network the machine is on.

```
(base) saad@Muhammads-MacBook-Air ~ % ifconfig en0
en0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST> mtu 1500
        options=6460<TSO4,TSO6,CHANNEL_IO,PARTIAL_CSUM,ZEROINVERT_CSUM>
        ether d6:e1:9c:e4:41:aa
        inet6 fe80::67:dfe6:afa4:fbdb%en0 prefixlen 64 secured scopeid 0xb
        inet6 2a00:f2a:e027:e570:102c:cdfa:4605:120 prefixlen 64 autoconf secured
        inet6 2a00:f2a:e027:e570:682c:ae8e:5b9b:cd8 prefixlen 64 autoconf temporary
        inet 10.219.1.71 netmask 0xffffff00 broadcast 10.219.1.255
        nd6 options=201<PERFORMNUD,DAD>
        media: autoselect
        status: active
```

Figure 9.2: Interface en0 showing IPv4 address 10.219.1.71 and IPv6 addresses

Field	Value and meaning
`inet`	`10.219.1.71`, my private IPv4 address (RFC 1918, Class A range)
`netmask`	`0xffffff00` = 255.255.255.0 = **/24**, so the network is 10.219.1.0/24 with 254 usable hosts
`broadcast`	`10.219.1.255`, the last address of the /24 network
`inet6`	One link-local address (`fe80::`) plus global IPv6 addresses (`2a00:...`), one of them marked temporary for privacy
`ether`	MAC address of the interface (Layer 2)
`mtu / status`	MTU 1500 bytes, status active

Step 3: View the routing table

```
netstat -rn -f inet
```

This prints the IPv4 routing table, which shows where my machine sends traffic and which other devices it has recently talked to.

```
(base) saad@Muhammads-MacBook-Air ~ % netstat -rn -f inet
Routing tables

Internet:
Destination        Gateway            Flags               Netif Expire
default            10.219.1.1         UGScg                 en0
10.219.1/24        link#11            UCS                   en0      !
10.219.1.1/32      link#11            UCS                   en0      !
10.219.1.1         80:69:1a:d3:66:c0  UHLWIir               en0   1171
10.219.1.71/32     link#11            UCS                   en0      !
10.219.1.128       5c:ea:1d:db:a4:49  UHLWI                 en0      !
10.219.1.143       0:5b:94:f2:4b:ad   UHLWI                 en0   1028
10.219.1.193       6c:1f:8a:d9:80:73  UHLWIi                en0   1057
10.219.1.255       ff:ff:ff:ff:ff:ff  UHLWbI                en0      !
127                127.0.0.1          UCS                   lo0
127.0.0.1          127.0.0.1          UH                    lo0
169.254            link#11            UCS                   en0      !
224.0.0/4          link#11            UmCS                  en0      !
224.0.0.251        1:0:5e:0:0:fb      UHmLWI                en0
255.255.255.255/32 link#11            UCS                   en0      !
(base) saad@Muhammads-MacBook-Air ~ %
```

Figure 9.3: IPv4 routing table of the lab machine

Entry	Meaning
`default via 10.219.1.1`	Default gateway (the router). Everything not on the local network goes here.
`10.219.1/24 (link#11)`	The directly connected local network on en0
`10.219.1.128 / .143 / .193`	Other devices on the LAN that my machine has talked to (their MAC addresses are cached)
`10.219.1.255`	Broadcast address of the local network
`127.0.0.1 (lo0)`	Loopback, the address that Step 1 scanned
`224.0.0.0/4`	Multicast range

Step 4: Identify the services with version detection

```
nmap -sV -p 3306,5000,5432,7000,8021 127.0.0.1
```

`-p` limits the scan to the five open ports and `-sV` asks Nmap to identify the real service behind each one. This took longer (**20.57 seconds**) because Nmap connects to every port and analyses the reply.

```
(base) saad@Muhammads-MacBook-Air ~ % nmap -sV -p 3306,5000,5432,7000,8021 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-16 16:20 +0400
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00029s latency).

PORT     STATE SERVICE    VERSION
3306/tcp open  mysql?
5000/tcp open  rtsp
5432/tcp open  postgresql PostgreSQL DB 9.6.0 or later
7000/tcp open  rtsp
8021/tcp open  tcpwrapped
3 services unrecognized despite returning data. If you know the service/version, please submit the following fingerprints at https://nmap.org/cgi-bin/submit.cgi?new-service :
==============NEXT SERVICE FINGERPRINT (SUBMIT INDIVIDUALLY)==============
SF-Port3306-TCP:V=7.98%I=7%D=10/16%Time=6AD216A8%P=arm-apple-darwin25.2.0%
SF:r(NULL,4D,"I\0\0\0\n9\.3\.0\0\x0b\0\0\0prj0R~\x7f,\0\xff\xff\xff\x02\0\
SF:xff\xdf\x15\0\0\0\0\0\0\0\0\0\0_\x173h}GF\x16\n\x1f3\x14\0caching_sha2_
SF:password\0*)%r(GenericLines,72,"I\0\0\0\n9\.3\.0\0\x0b\0\0\0prj0R~\x7f,
SF:\0\xff\xff\xff\x02\0\xff\xdf\x15\0\0\0\0\0\0\0\0\0\0_\x173h}GF\x16\n\x1
SF:f3\x14\0caching_sha2_password\0!\0\0\x01\xff\x84\x04#08S01Got\x20packet
SF:s\x20out\x20of\x20order*)%r(GetRequest,72,"I\0\0\0\n9\.3\.0\0\x0c\0\0\0
```

Figure 9.4: Version detection (-sV) on the five open ports

Nmap printed “3 services unrecognized despite returning data” followed by long fingerprint blocks. This is not an error. It means those services replied but did not match Nmap’s database, so Nmap offers the fingerprint for submission. The fingerprint for port 3306 contains the text `caching_sha2_password`, which is part of a **MySQL 8** login handshake, so the “mysql?” guess is correct.

Step 5: Find which program owns each port

```
sudo lsof -nP -iTCP:3306 -iTCP:5000 -iTCP:5432 -iTCP:7000 -iTCP:8021 -sTCP:LISTEN
```

Nmap tells me that a port is open, but not which program opened it. `lsof` lists open sockets together with the process name, process ID (PID), user and the address the socket is bound to. `sudo` is needed to see processes owned by other users such as root. The flags are: `-n` no DNS lookups, `-P` show port numbers, `-iTCP:<port>` filter by TCP port and `-sTCP:LISTEN` show only listening sockets.

```
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 20.57 seconds
(base) saad@Muhammads-MacBook-Air ~ % sudo lsof -nP -iTCP:3306 -iTCP:5000 -iTCP:5432 -iTCP:7000 -iTCP:8021 -sTCP:LISTEN
Password:
COMMAND    PID    USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
launchd      1    root   31u  IPv6 0x7b225f9317b7c1aa      0t0  TCP [::1]:8021 (LISTEN)
launchd      1    root   32u  IPv4 0x79287d0cb7a01108      0t0  TCP 127.0.0.1:8021 (LISTEN)
launchd      1    root   34u  IPv6 0x7b225f9317b7c1aa      0t0  TCP [::1]:8021 (LISTEN)
launchd      1    root   35u  IPv4 0x79287d0cb7a01108      0t0  TCP 127.0.0.1:8021 (LISTEN)
mysqld     528 _mysql   21u  IPv6  0xc2627c531f16b05      0t0  TCP *:3306 (LISTEN)
ControlCe 680    saad    9u  IPv4 0x732bd06038ce581f      0t0  TCP *:7000 (LISTEN)
ControlCe 680    saad   10u  IPv6 0x79115c069f3ef21c      0t0  TCP *:7000 (LISTEN)
ControlCe 680    saad   11u  IPv4 0xc56d707a30b3d917      0t0  TCP *:5000 (LISTEN)
ControlCe 680    saad   12u  IPv6 0x7442998c4fc37fa8      0t0  TCP *:5000 (LISTEN)
postgres  993    saad    7u  IPv6 0x853bce8c98d0ac8a      0t0  TCP [::1]:5432 (LISTEN)
postgres  993    saad    8u  IPv4 0xc2f63b2b8b3894d1      0t0  TCP 127.0.0.1:5432 (LISTEN)
(base) saad@Muhammads-MacBook-Air ~ %
```

Figure 9.5: lsof output linking each open port to its process, user and bind address

8.5 Results: Port-by-Port Analysis

Port	Quick scan guess	Version scan (-sV)	Process (PID, user)	Bound to	Who can reach it
3306	mysql	mysql? (MySQL 8 handshake in fingerprint)	mysqld (528, _mysql)	* (all interfaces)	Any device on the network, unless a firewall blocks it
5000	upnp	rtsp	ControlCenter (680, saad)	* (all interfaces)	Any device on the network
5432	postgresql	PostgreSQL 9.6.0 or later	postgres (993, saad)	127.0.0.1 and ::1	This machine only
7000	afs3-fileserver	rtsp	ControlCenter (680, saad)	* (all interfaces)	Any device on the network
8021	ftp-proxy	tcpwrapped	launchd (1, root)	127.0.0.1 and ::1	This machine only

8.6 Analysis: What the Results Show

- **Port-based guesses can be wrong.** The first scan called port 5000 “upnp” and port 7000 “afs3-fileserver” only because of the port numbers. Version detection and `lsof` showed that both belong to the macOS **ControlCenter** process, and Nmap saw an RTSP-style reply. ControlCenter is the macOS process behind the **AirPlay Receiver** feature, which uses these two ports.

- **Scanning 127.0.0.1 does not show network exposure.** Loopback scans report every local listener as open, including ones that outsiders can never reach. The **Bound to** column from `lsof` is what separates the two groups. `127.0.0.1` / `::1` means local only. `*` means every network interface, including Wi-Fi at 10.219.1.71.
- **Local only (lower risk):** PostgreSQL on 5432 and the launchd-managed service on 8021. Port 8021 is held by `launchd` (PID 1) on behalf of a macOS ftp-proxy service, which is why Nmap showed it as "tcpwrapped": the connection is accepted and then closed without talking.
- **Listening on all interfaces (higher risk):** MySQL on 3306 and ControlCenter on 5000 and 7000. A database listening on every interface is the most important finding, because database servers are a common target for password guessing.
- **Same question, different tool.** `lsof` on macOS plays the role of `ss -tuln` on Linux (Task 3.10): it lists listening sockets from inside the machine, while Nmap shows what answers from outside the process.

OSI layer	Where it appeared in this task
Layer 7 Application	Service and version detection (-sV): MySQL, PostgreSQL, RTSP-style replies
Layer 4 Transport	TCP ports and their states (open, closed); the 3306, 5432 and other port numbers
Layer 3 Network	IP address 10.219.1.71/24, default gateway 10.219.1.1, routing table
Layer 2 Data Link	MAC addresses of the interface and of neighbouring devices in the routing table

8.7 Security Observations and Hardening

- **MySQL (3306):** change it to listen on localhost only by setting `bind-address = 127.0.0.1` in the MySQL configuration, or stop the service when it is not needed (for example `brew services stop mysql` if it was installed with Homebrew).
- **AirPlay Receiver (5000 and 7000):** turn it off in System Settings if I do not stream to this Mac. This closes both ports.
- **Firewall:** enable the macOS firewall so that unwanted inbound connections are blocked on public or shared Wi-Fi.
- **Keep local-only services local:** PostgreSQL and the ftp-proxy listener are already bound to loopback, which is the safe default.
- **Verify from the outside:** this task scanned only localhost. A stronger test is to scan my own address (`nmap 10.219.1.71`) from a second device I own on the same network, which shows exactly what other devices can reach.

8.8 Findings Summary

Item	Result
Target	My own machine, 127.0.0.1 (LAN address 10.219.1.71/24, gateway 10.219.1.1)
Scan time	Default scan 0.10 s; version scan 20.57 s
Ports scanned	Top 1000 TCP ports: 5 open, 995 closed
Open ports	3306 (MySQL), 5000 and 7000 (AirPlay, via ControlCenter), 5432 (PostgreSQL), 8021 (ftp-proxy via launchd)
Exposed to the network	Ports 3306, 5000 and 7000 (bound to all interfaces)

Item	Result
Local only	Ports 5432 and 8021 (bound to 127.0.0.1 and ::1)
Key lesson	Nmap shows what is open; lsof shows who owns it and where it is bound. Both are needed to judge real exposure.
Next step	Fix the MySQL bind address, disable AirPlay Receiver, then re-scan from a second device to confirm

Task 9: Wazuh SIEM Deployment (Proposed Design)

Status of this section

This section is a design and planning document. The Wazuh deployment described here was NOT carried out, because the local machine did not have enough free storage to host the required virtual machine. No screenshots, alerts or output values are included, and none have been invented. Everything under "Expected" describes behaviour documented for Wazuh, not results observed by me.

9.1 Objective

To plan the deployment of Wazuh, an open-source SIEM and XDR platform, and to define how it would be configured and tested to collect logs, detect suspicious activity and raise alerts from a monitored endpoint. This links to earlier tasks in this report: the network traffic studied in Tasks 4 to 6 and the Linux commands in Task 3 are the kind of activity a SIEM would monitor.

9.2 What is Wazuh?

Wazuh is a free, open-source security platform that combines SIEM (Security Information and Event Management) and XDR (Extended Detection and Response) capabilities. Software agents installed on endpoints send log and system data to a central manager, which analyses it against a rule set, and alerts are stored and displayed on a web dashboard. Its main capabilities are:

- **Log data analysis:** collects and decodes logs from the OS and applications, then matches them against detection rules.
- **File integrity monitoring (FIM):** detects creation, modification or deletion of monitored files and directories.
- **Vulnerability detection:** compares installed software against known vulnerability (CVE) data.
- **Security configuration assessment (SCA):** checks hosts against benchmarks such as CIS.
- **Intrusion and rootkit detection:** looks for hidden processes, suspicious files and anomalies.
- **Active response:** can automatically act on an alert, for example by blocking an IP address.

9.3 Architecture

Wazuh has three central components plus the agents on monitored machines.

Component	Role	Default port(s)
Wazuh agent	Runs on the monitored endpoint; collects logs, file changes and system inventory and forwards them securely.	1514/TCP (to manager), 1515/TCP (enrolment)
Wazuh server (manager)	Receives agent data, decodes events, applies rules and generates alerts.	1514, 1515, 55000/TCP (API)
Wazuh indexer	Stores and indexes alerts and events so they can be searched (OpenSearch-based).	9200/TCP
Wazuh dashboard	Web interface for viewing alerts, agents, vulnerabilities and compliance data.	443/TCP (HTTPS)

Data flow: endpoint logs and events, then the agent, then the server (decode and rule match), then the indexer (storage), then the dashboard (visualisation and alerts).

9.4 Proposed Lab Environment

Item	Planned specification
Wazuh server host	Ubuntu Server 22.04 LTS, 2 vCPU, 4 GB RAM minimum (8 GB preferred), 20 GB or more disk. All-in-one install (server, indexer and dashboard on one host).
Monitored endpoint	A second Ubuntu VM (or the Task 3 Linux environment) running the Wazuh agent.
Attacker / test source	A third VM or the host machine, used only to generate failed SSH logins against the lab endpoint.
Network	All machines on one isolated host-only or private virtual network, so tests never touch other systems.
Where it would run	A hypervisor (VirtualBox/VMware) on a machine with enough free disk, or a cloud VM. Not done here due to storage limits.

9.5 Planned Deployment Steps

Step 1: Install the server. Wazuh provides an assisted installer for the all-in-one setup. The exact version in the URL should be checked on the official Wazuh documentation before use.

```
curl -sO https://packages.wazuh.com/4.x/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

The installer prints the dashboard admin credentials at the end, which must be saved.

Step 2: Verify the services.

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard
```

Step 3: Open the dashboard. Browse to https://<server-ip> and log in as the admin user.

Step 4: Add an agent. In the dashboard, use "Deploy new agent", choose the OS, enter the server IP, and run the generated command on the endpoint. Then start the agent service:

```
sudo systemctl daemon-reload
sudo systemctl enable --now wazuh-agent
```

Step 5: Confirm the agent. The agent should appear as "Active" in the Agents page.

9.6 Planned Use Cases and Tests

Use case	Test to perform	Expected behaviour (per Wazuh documentation)
SSH brute-force detection	Attempt several failed SSH logins from the test source to the endpoint.	Authentication-failure alerts from the sshd rules; repeated failures may trigger a higher-level brute-force alert.
File integrity monitoring	Create, edit and delete a file in a monitored directory such as /etc or a test folder.	Syscheck alerts showing the file path and the type of change.
Vulnerability detection	Enable the module and let the agent report its installed packages.	A list of detected CVEs per package with severity levels.
Security configuration assessment	Review the default SCA scan on the endpoint.	A policy score with passed and failed checks.

Use case	Test to perform	Expected behaviour (per Wazuh documentation)
Privilege escalation logging	Run a sudo command and a failed sudo attempt.	Alerts for sudo usage and authentication failure.
Active response (optional)	Enable a firewall-drop response for repeated SSH failures.	The offending test IP is blocked temporarily.

9.7 Evidence to Collect Once Deployed

If the deployment is completed later, the following evidence would show the work was done. These are listed as a plan only and are not included in this report.

- Dashboard home page showing the server running.
- Agents page showing the endpoint as Active.
- Alerts for the failed SSH logins, with timestamps and rule descriptions.
- A file integrity monitoring alert for the test file change.
- The vulnerability detection and SCA summary pages.

9.8 Limitations

- **Not implemented:** the deployment was not run, so there are no observed results and I make no claims about real alert counts or timings.
- **Resource needs:** Wazuh needs several GB of RAM and tens of GB of disk. The available local storage was insufficient.
- **Lab scope:** even when deployed, a single-agent lab does not show how Wazuh performs at scale or against real attacker traffic.
- **Version differences:** commands and dashboard menus may vary slightly between Wazuh versions.

9.9 Findings Summary

Area	Summary
Tool	Wazuh, an open-source SIEM/XDR platform.
Architecture	Agents, server, indexer and dashboard.
Planned tests	SSH brute force, FIM, vulnerability detection, SCA, sudo logging.
Status	Designed and documented only; not deployed.
Value to this report	Shows how network and OS knowledge from Tasks 1 to 6 would be applied to centralised monitoring and detection.

Conclusion

This report covered the main building blocks of operating systems and networking, and applied them to hands-on security work. Each task added a layer of understanding, moving from theory to practical tools:

- **OSI model (Task 1):** explained how the seven layers divide network communication, with the protocols, devices and encapsulation at each layer.
- **Subnetting (Task 2):** calculated subnet masks, network and broadcast addresses, host ranges and VLSM, which are needed for planning and segmenting networks.
- **Linux commands (Task 3):** demonstrated ten essential commands for navigation, file handling, permissions, searching, and process and network inspection.
- **DNS and HTTP in Wireshark (Task 4):** captured and identified DNS (UDP/53) and HTTP (TCP/80) packets and mapped them to OSI layers.
- **Python sniffer (Task 5):** built a Scapy packet sniffer in Kali Linux that shows the source, destination, protocol, ports and size of live packets.
- **Packet Tracer home network (Task 6):** designed a star-topology home network, planned its addressing, traced traffic flow and identified security weaknesses.
- **Traffic analysis (Task 7):** analysed the TCP three-way handshake, DNS queries and responses, and ICMP ping traffic, and compared which protocols are readable and which are encrypted.
- **Nmap scan (Task 8):** scanned my own lab machine to see its open ports the way an attacker would, then used version detection and system commands to confirm which services were behind them.
- **Wazuh SIEM (Task 9):** documented a proposed Wazuh deployment, including its architecture, lab design, planned tests and expected behaviour. This task was a design only and was not deployed because of limited local storage, so it contains no observed results.

Key takeaways. First, a lot of everyday traffic, such as DNS and plain HTTP, is readable by anyone who can capture it, which is why encryption matters. Second, every machine exposes an attack surface through its open ports, so knowing what is listening is the first step in securing it. Third, tools such as Wireshark, Scapy and Nmap show what is happening on a network, while a SIEM such as Wazuh would bring that visibility together for continuous monitoring and alerting.

Limitations. All practical work was done in small lab environments on my own machines, so the results show how the tools work rather than how they perform on large or live production networks. Task 9 remains to be tested in a real deployment.

Overall, these skills are foundational for cybersecurity work such as network design, traffic analysis, penetration testing, monitoring and incident response, and they give me a base to build on in later sessions..